

OST
Ostschweizer
Fachhochschule

HackApertus · Track 2A: Q&A

Multilingual Natural Language Inference over Swiss Official Voting Booklets

Evaluation, starter repo and demo app

Prof. Dr. Mitra Purandare & Abinas Kuganathan

08 Oct 2026 · 12:00–12:30 CEST

OST - Ostschweizer Fachhochschule, Campus Rapperswil

Agenda

01 The challenge in two minutes

02 Starter repo and demo app

03 Evaluation and scoring

04 Your questions



The challenge

What your solution has to do

OST - Ostschweizer Fachhochschule, Campus Rapperswil

Prof. Dr. Mitra Purandare & Abinas Kuganathan

Generative AI

08.10.2026

OST
Ostschweizer
Fachhochschule

Check claims against Swiss voting booklets

Your solution gets a **claim** and a **source** (a booklet PDF or a passage) and returns:

- a **label**: does the source support, refute or say nothing about the claim?
- **evidence**: verbatim quotes, with PDF page for task A
- **metrics**: tokens and time per case

> Judge against the **supplied source**, not truth in the world.

Dataset: `OSTswiss/MNLIoverSwissVotingBooklets` (public, about 1,500 rows, balanced over labels and all DE/FR/IT pairs). Final scoring uses a **held-out private set**.

Label	Name	Source …
`0`	entailment	supports the claim
`1`	neutral	has too little information
`2`	contradiction	refutes the claim

Cross-lingual

Claim and source language can differ. All 9 DE/FR/IT combinations must work.

Two tasks, one format

Task A · Document

Advanced · 60 %

Input: booklet PDF + vote name + claim

- Find the relevant passage yourself (parsing, retrieval, compression …), then do NLI
- Labels 0 and 2 **need evidence** with a 1-based page
- Start with a **full-document baseline**; more complex pipelines must earn their keep

Task B · Reference

Beginner · 40 %

Input: reference passage + vote name + claim

- Pure NLI, no retrieval
- Evidence is optional and not scored
- For neutral cases the passage is real but **unrelated**: its presence says nothing about the label

- Each request has **exactly one** of `booklet` (A) or `reference` (B); input files can mix both.
- Use `vote` to find the right proposal: a booklet usually covers several votes.



Starter repo & demo app

gitlab.com/ifsoftware/hackapertus-starter

OST - Ostschweizer Fachhochschule, Campus Rapperswil

Prof. Dr. Mitra Purandare & Abinas Kuganathan

Generative AI

08.10.2026

OST
Ostschweizer
Fachhochschule

What is in the repo

File	Purpose
`main.py`	Demo app: the prediction CLI. Returns **random labels**, empty evidence, zero metrics.
`prepare_cases.py`	Pulls `v1.1.jsonl` from Hugging Face and writes `data/cases.jsonl` + `data/expected-labels.jsonl`.
`evaluate.py`	Scores predictions locally with the official rules (as far as possible offline).
`Dockerfile`	`python:3.12-slim` + uv; entrypoint is the CLI.
`examples/`	One real task A and one task B request, plus matching output.

Optional, not required

Use any language and tooling. Only the **Docker + CLI contract** is fixed.

Never pass the gold labels in

`expected-labels.jsonl` is for local scoring only. Keep it outside the prediction container.

Run the demo app

1 · Run the example

```
uv sync --locked
uv run python main.py \
  --input examples/cases.jsonl \
  --output output/predictions.jsonl
```

2 · Generate real cases

```
# all rows: one A and one B case each
uv run python prepare_cases.py
# 10 rows + task A PDFs
uv run python prepare_cases.py \
  --limit 10 --download-booklets
# task B only
uv run python prepare_cases.py --task B
```

3 · Score it

```
uv run python evaluate.py \
  --predictions output/predictions.jsonl \
  --expected data/expected-labels.jsonl \
  --cases data/cases.jsonl \
  --json output/score.json
```

> Random labels give macro-F1 around **0.33**. That is the floor your solution has to beat by a lot.

- `--limit N` takes the **first** N rows: not balanced across labels
- IDs use row positions and can change when the dataset changes

Request: what your CLI reads

Task A · Document PDF path, you find the passage

```
{
  "id": "v1.0-row-1158-A",
  "booklet": {
    "path": "booklets/2024_09_22_de.pdf",
    "language": "de"
  },
  "vote": "Volksinitiative «Für die Zukunft
           unserer Natur und Landschaft
           (Biodiversitätsinitiative)»",
  "claim": {
    "text": "Laut dem Abstimmungstext stammt
             der Bundesbeschluss über die
             Volksinitiative … vom 15. Juni 2024.",
    "language": "de"
  }
}
```

Task B · Reference passage given, claim in Italian

```
{
  "id": "v1.0-row-414-B",
  "reference": {
    "text": "Volksinitiative «Für die Zukunft …»
             Abstimmungstext Bundesbeschluss über
             die Volksinitiative … vom 22. Dezember
             2023",
    "language": "de"
  },
  "vote": "Volksinitiative «Für die Zukunft
           unserer Natur und Landschaft
           (Biodiversitätsinitiative)»",
  "claim": {
    "text": "Il testo di voto afferma che il
             Bundesbeschluss … è datato
             22 dicembre 2023.",
    "language": "it"
  }
}
```

Request fields

Field	Contents
`id`	Unique string, echo unchanged
`vote`	Vote name in the **source** language: picks the right section of the booklet
`claim`	`text` + `language` (`de` / `fr` / `it`), can differ from the source
`booklet` (A)	`path` relative to `/data` + `language`
`reference` (B)	`text` + `language`

- Exactly **one** of `booklet` or `reference` per request; files can mix both
- `booklets/2024_09_22_de.pdf` → `/data/booklets/2024_09_22_de.pdf`
- Cases share PDFs: caching is fine, but results must **not depend on case order**
- The gold label is never sent; in task A the gold passage is not sent either

Response: what your CLI writes

```
{
  "id": "v1.1-row-60-A",
  "label": 0,
  "label_name": "entailment",
  "evidence": [{
    "page": 43,
    "text": "Per tutte queste ragioni,
             Consiglio federale e Parlamento
             raccomandano di accettare …"
  }],
  "metrics": {
    "input_tokens": 1304,
    "output_tokens": 46,
    "inference_time_ms": 776
  }
}
```

- **One line per input** `id`, any order, exit code `0`
- Missing or invalid lines count as **wrong**; `label_name` must match `label`
- **Evidence (A, labels 0/2):** at least one item, **1-based** page, **verbatim** in the source language (claim was German, quote stays Italian)
- Each item at most about one page (5,000 chars); only the **first five** count; most relevant first
- **Task B:** `"page": null` or `"evidence": []`
- **Metrics:** sum over all LLM calls in the case, output tokens include reasoning

Docker: how we run your solution

```
docker build --platform linux/amd64 -t my-solution:dev .
docker run --rm --platform linux/amd64 \
  -e BASE_URL=https://api.inference.cscs.ch/v1 \
  -e API_KEY="$CSCS_API_KEY" \
  -v "$PWD/data/cases.jsonl:/data/cases.jsonl:ro" \
  -v "$PWD/data/booklets:/data/booklets:ro" \
  -v "$PWD/output:/output" \
  my-solution:dev \
  --input /data/cases.jsonl \
  --output /output/predictions.jsonl
```

At evaluation, `BASE_URL` points to our **token-counting proxy** and `API_KEY` is your team key. Same variable names.

- `linux/amd64`, **no GPU**
- `/data` is read-only; write to `/output`, caches to `/tmp`
- **Bake in** dependencies and local model weights; nothing is downloaded at runtime
- **No secrets** in the image: no API keys, no `.env`
- Only **Apertus v1.5** (`swiss-ai/Apertus-v1.5-…`), every remote call through `BASE_URL`
- Local parsing, OCR, embeddings, BM25 are fine; document them in the report

Submission: repo + published container

1. Build your solution in a repo on **GitHub** or **GitLab**
2. Build the image for `linux/amd64`
3. **Publish the container** to the registry of your repo: GitHub Container Registry or GitLab Container Registry
4. We pull your image and run it with the command from the previous slide

> Make sure we can pull the image, and keep secrets out of it: `BASE_URL` and `API_KEY` are injected at run time.

GitHub (`ghcr.io`)

```
docker login ghcr.io -u <user>
docker build --platform linux/amd64 \
  -t ghcr.io/<user>/<repo>:latest .
docker push ghcr.io/<user>/<repo>:latest
```

GitLab (`registry.gitlab.com`)

```
docker login registry.gitlab.com
docker build --platform linux/amd64 \
  -t registry.gitlab.com/<group>/<project>:latest .
docker push registry.gitlab.com/<group>/<project>:latest
```

Building and pushing from CI (GitHub Actions, GitLab CI) works too.

From random labels to a real solution

Read the endpoint **at runtime**. Environment variables win over `.env`:

```python
import os
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()  # does not override set vars
client = OpenAI(
    base_url=os.environ.get(
        "BASE_URL",
        "https://api.inference.cscs.ch/v1"),
    api_key=os.environ["API_KEY"],
)
```

Suggested path

1. Task B: one prompt with reference + claim + vote
2. Task A: full-document baseline
3. Measure tokens and time with `evaluate.py`
4. Then try retrieval, page selection, compression
5. Report what helped, with numbers

> Count tokens from `response.usage` and add them up over all calls in a case.



Evaluation

How a submission is scored

OST - Ostschweizer Fachhochschule, Campus Rapperswil

Prof. Dr. Mitra Purandare & Abinas Kuganathan

Generative AI

08.10.2026

OST
Ostschweizer
Fachhochschule

Qualify first, then score

Minimum macro-F1

0.60
Task A · Advanced

0.75
Task B · Beginner

> Below either threshold, the **whole submission** is invalid, not just that task.

Component		Task A	Task B
NLI macro-F1 (label only)	↑	50 %	70 %
Evidence (Hit@5)	↑	20 %	–
Processing time (non-LLM)	↓	15 %	15 %
Token usage (proxy)	↓	15 %	15 %

$$\textbf{Score} = 0.4 \cdot S_B + 0.6 \cdot S_A$$

↑ proportional: score = measured value, higher is better.
↓ inverse: score = best ÷ own, lower is better.

Tokens and time: relative to the best team

K_t = best tokens$_t$ ÷ own tokens$_t$
T_t = best time$_t$ ÷ own time$_t$

- **Lower is better.** Twice the tokens of the best team → **50 %**, four times → **25 %**
- Best = lowest value among teams with **≥ 0.65 macro-F1** on that task, so cheap but wrong cannot win
- No floor: best 0.3 s vs. your 3 s → **10 %**

Tokens per case

Counted by the **proxy**. Self-reported `metrics` are not used. Preprocessing tokens (e.g. summarising booklets) are included and spread over all cases.

Processing time per case

Wall-clock run time **minus** time with at least one LLM request in flight. Apertus latency does not count against you; parsing and indexing do.

Task A and task B run in separate containers. You cannot compute these two scores locally; report your own numbers anyway.

Evidence matching (task A)

For every gold entailment/contradiction case, it counts if **any of your first five** items:

- is at most 5,000 characters (about one page), and
- lies inside the gold passage **or contains it**

Text is normalised (Unicode, case, whitespace, line-break hyphens), then compared with `rapidfuzz.fuzz.partial_ratio`. **≥ 90** counts.

$$E_A = \text{cases found} \div \text{gold 0/2 cases}$$

This is **Hit@5** (Success@5): position within the first five does not matter.

Tips

- Quote from the section that treats the vote **in detail**, not only the summary at the front
- Fact appears twice? Cite **both** places as separate items
- A whole page as one item is fine
- Never translate the quote

Page numbers are not checked by `evaluate.py` yet. Give correct 1-based pages anyway.

Worked example

	Beginner (B)	Advanced (A)
Macro-F1	0.78	0.68
Evidence	—	0.50
Tokens (own / best)	1,990 / 1,700 → **0.85**	12,000 / 8,000 → **0.67**
Time (own / best)	20 ms / 10 ms → **0.50**	3 s / 0.3 s → **0.10**
Task score	**0.75**	**0.56**

$S_B = 0.70 \cdot 0.78 + 0.15 \cdot 0.50 + 0.15 \cdot 0.85$

$S_A = 0.50 \cdot 0.68 + 0.20 \cdot 0.50 + 0.15 \cdot 0.10 + 0.15 \cdot 0.67$

Final = $0.40 \cdot 0.75 + 0.60 \cdot 0.56$

0.64 (64 %)

Common mistakes to avoid

Contract

- A missing `id`, or a crash before all lines are written
- `label_name` does not match `label`
- Non-zero exit code
- Writing into `/data`, or reading `expected-labels.jsonl`
- Results that depend on case order

Docker and model

- Hard-coded URL or key, or a call that bypasses `BASE_URL`
- Models downloaded at runtime
- `.env` or keys baked into the image
- Built for `arm64` on a Mac without `--platform linux/amd64`
- A model other than Apertus v1.5
- Image not published, or not pullable by us

Evidence: translated quotes, 0-based pages, items over 5,000 chars, best quote not in the first five.



Questions?

OST

Guide	hackapertus.notion.site · Solution API Guide
Starter	gitlab.com/ifsoftware/hackapertus-starter
Dataset	huggingface.co/datasets/OSTswiss/MNLIoverSwissVotingBooklets